

Git Branching: Create, Switch, Merge & Delete

Introduction

Git branches allow you to work on different features or fixes independently without affecting the main codebase. This is an essential skill for DevOps and collaborative development.

 **Watch the Video:**

<https://youtu.be/HQllaDP7k0E?si=vPKe5IM7I-35DLgg>

What You Will Learn

- What Git branches are and why they are used
- How to create a new branch
- How to switch between branches
- How to merge branches
- How to delete branches safely
- Common Git branch commands used in real projects

Creating a Branch

```
git branch <branch-name>
```

Example:

```
git branch feature-login
```

Switching Branch

```
git checkout <branch-name>
```

or (modern way):

```
git switch <branch-name>
```

Create and Switch at the Same Time

```
git checkout -b <branch-name>
```

or

```
git switch -c <branch-name>
```

Merge Branch

First switch to main branch:

```
git switch main
```

Then merge:

```
git merge <branch-name>
```

Delete Branch

Delete safely (if merged)

```
git branch -d <branch-name>
```

Force delete (if not merged)

```
git branch -D <branch-name>
```

Rename Branch

```
git branch -M <new-name>
```

Key Points

- Branches help you work on features separately
- Always merge carefully into `main` or `master`
- Use `switch` instead of `checkout` in modern Git
- Delete unused branches to keep repo clean
- Always test before merging code

Simple Workflow

```
Create branch → Work → Commit → Switch to main → Merge → Delete branch
```